University of Colombo School of Computing

SCS 1308 - Foundations of Algorithms

Tutorial - Recurrence Relations

1. Recurrence relation

Given:

- Divide the dataset into two equal parts.
- Dividing cost is negligible.
- Process both subproblems recursively.
- Combining the results of a problem of size n requires $3n + 5$ operations.
- Base case ($n = 1$) requires 4 operations.
- Two recursive calls execute sequentially and their running times are added.

Recurrence relation:

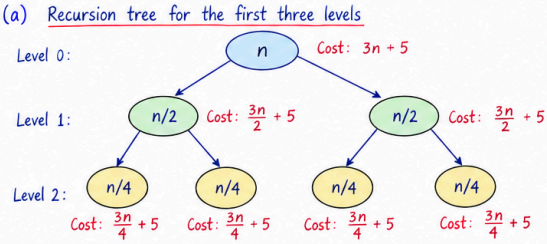
$$T(n) = 2T\left(\frac{n}{2}\right) + 3n + 5 \quad (n > 1)$$
$$T(1) = 4$$

Explanation:

- $2T(n/2)$: there are two recursive calls on subproblems of size $\frac{n}{2}$, (their running times are added since they execute sequentially).
- $3n + 5$: additional work to combine the results of size n.
- $T(1) = 4$: base case cost for a problem of size 1.

2. Recursion Tree Method

(a) Recursion tree for the first three levels



(b) Number of nodes and total non-recursive cost at level i

Level i	Number of nodes	Problem size per node	Cost per node	Total cost at level i
i	2^i	$\frac{n}{2^i}$	$\frac{3n}{2^i} + 5$	$2^i\left(\frac{3n}{2^i} + 5\right) = 3n + 5 \cdot 2^i$

(c) Height of the tree and total cost of internal levels

- Tree height : $\frac{n}{2^p} = 1 \Rightarrow p = \log_2 n$ (since n is a power of 2)
- Total cost of internal levels (levels 0 to $p-1$) :

$$I = \sum_{i=0}^{p-1} (3n + 5 \cdot 2^i) \quad \text{(from part (b))}$$
$$= 3np + 5(2^p - 1)$$
$$= 3n \log_2 n + 5(n - 1)$$
$$= 3n \log_2 n + 5n - 5$$

(d) Leaf cost and overall time complexity

- Number of leaves : $2^p = n$
- Cost per leaf : $T(1) = 4$
- Total leaf cost : $4n$

Total running time:

$$T(n) = (3n \log_2 n + 5n - 5) + 4n$$
$$= 3n \log_2 n + 9n - 5$$
$$T(n) = \Theta(n \log n)$$

3. Iteration Method

Given recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + 3n + 5 \quad (n > 1)$$
$$T(1) = 4$$

(a) Expand the recurrence at least three times

1st expansion:

$$T(n) = 2T\left(\frac{n}{2}\right) + 3n + 5 \quad (1)$$

2nd expansion:

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + 3\left(\frac{n}{2}\right) + 5\right] + 3n + 5$$
$$= 4T\left(\frac{n}{4}\right) + 6n + 15 \quad (2)$$

3rd expansion:

$$T(n) = 4\left[2T\left(\frac{n}{8}\right) + 3\left(\frac{n}{4}\right) + 5\right] + 6n + 15$$
$$= 8T\left(\frac{n}{8}\right) + 9n + 35 \quad (3)$$

(b) General pattern after k expansions

From the above expansions, we can observe the pattern:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + 3nk + 5\left(2^k - 1\right) \quad (4)$$

(c) Determine the correct value of k

The recursion stops when the problem size becomes 1.

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$

(d) Obtain a closed-form expression for $T(n)$

Substitute $k = \log_2 n$ into (4):

$$T(n) = 2^{\log_2 n} T(1) + 3n(\log_2 n) + 5\left(2^{\log_2 n} - 1\right)$$
$$= n \cdot 4 + 3n \log_2 n + 5(n - 1)$$
$$= 4n + 3n \log_2 n + 5n - 5$$
$$T(n) = 3n \log_2 n + 9n - 5$$

(e) Asymptotic time complexity

The dominant term is $3n \log_2 n$. Therefore,

$$T(n) = \Theta(n \log n)$$

4. Substitution Method

Recurrence relation (from part 1):

$$T(n) = 2T\left(\frac{n}{2}\right) + 3n + 5 \quad (n > 1)$$
$$T(1) = 4$$

(a) Induction hypothesis

We prove that $T(n) = O(n \log n)$.

Assume that for some constant $c > 0$ and for all $n \geq 2$,

$$T(n) \leq c\, n \log_2 n \qquad (1)$$

We will show that the above assumption implies $T(n) \leq c\, n \log_2 n$ for $n \geq 2$.

(b) Apply the hypothesis to the recurrence

For $n \geq 2$,

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + 3n + 5 \\ &\leq 2\left[c\left(\frac{n}{2}\right)\log_2\left(\frac{n}{2}\right)\right] + 3n + 5 \quad \text{(by (1))} \\ &= cn\log_2\left(\frac{n}{2}\right) + 3n + 5 \\ &= cn(\log_2 n - 1) + 3n + 5 \\ &= cn\log_2 n - cn + 3n + 5 \end{aligned}$$

(c) Show that the inequality holds

We need to show that

$$cn\log_2 n - cn + 3n + 5 \leq cn\log_2 n$$

Cancel the common term $cn\log_2 n$ from both sides:

$$-cn + 3n + 5 \leq 0$$
$$\Rightarrow (c-3)n \geq 5$$

Choose a constant $c \geq 10$. Then $(c-3) = 7$, and $7n \geq 5$ which is true for all $n \geq 2$.

Hence, for $c = 10$ (or any $c \geq 10$),

$$T(n) \leq cn\log_2 n \quad \text{for all } n \geq 2$$

(d) Verify the base case

For $n = 1$,

$$T(1) = 4 \quad \text{and} \quad c \cdot 1 \cdot \log_2 1 = 0$$

So, $T(1) \leq c \cdot 1 \cdot \log_2 1 + 4$ holds for any $c \geq 0$.

Hence, the base case is satisfied (or we can start the induction from $n = 2$ as shown above).

Conclusion

By mathematical induction, $T(n) = O(n \log n)$.

From the exact expression obtained in part 3, $T(n) = 3n\log_2 n + 9n - 5$, we also have $T(n) = \Omega(n \log n)$.

Therefore, $\boxed{T(n) = \Theta(n \log n)}$

5. Master Theorem

Given recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + 3n + 5 \quad (n > 1)$$
$$T(1) = 4$$

(a) Identify a, b and f(n)

The Master Theorem is applicable to recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

Comparing with the given recurrence,

$a = 2$ (two recursive calls)

$b = 2$ (problem size is divided by 2)

$f(n) = 3n + 5$ (non-recursive work)

(b) Calculate $n^{\log_b a}$

$$\begin{aligned} n^{\log_2 a} &= n^{\log_2 2} \\ &= n^1 \end{aligned}$$

$$\boxed{n^{\log_b a} = n}$$

(c) Identify the applicable case

The Master Theorem has three cases. We compare $f(n)$ with $n^{\log_b a} = n$.

Given $f(n) = 3n + 5$

Since $3n + 5 = \Theta(n)$, we have

$$f(n) = \Theta(n) = \Theta(n^{\log_2 a})$$

Therefore, this is **Case 2** of the Master Theorem.

(d) State the final asymptotic time complexity

For Case 2, the Master Theorem gives

$$T(n) = \Theta(n^{\log_b a} \log n)$$

Substituting $n^{\log_b a} = n$,

$$\boxed{T(n) = \Theta(n \log n)}$$

(e) Compare with other methods

The time complexity obtained using the Master Theorem is consistent with the results from the other methods:

- Recursion tree method : $\Theta(n \log n)$
- Iteration method : $\Theta(n \log n)$
- Substitution method : $\Theta(n \log n)$

Hence, all methods give the same asymptotic time complexity.

6. Asymptotic Complexity Graph

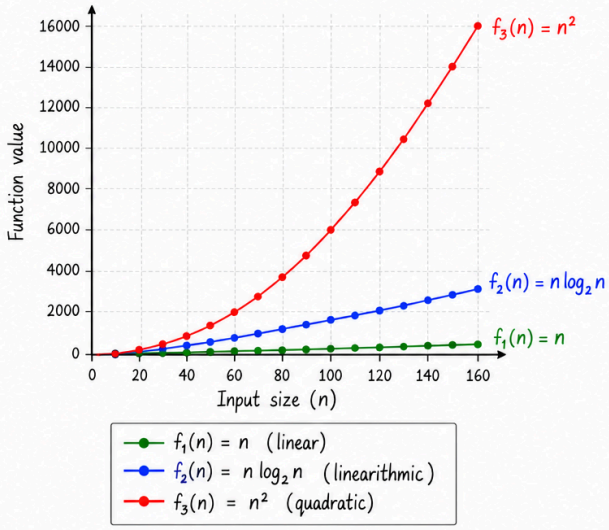
Consider the following functions:

$f_1(n) = n$

$f_2(n) = n \log_2 n$

$f_3(n) = n^2$

(a) Draw the graph of the three functions



(b) Identify the curve for the algorithm

From parts 2 – 5, the time complexity of the recursive algorithm is

$$T(n) = \Theta(n \log n)$$

Among the given functions, this corresponds to

$$f_2(n) = n \log_2 n$$

Therefore, the curve $f_2(n) = n \log_2 n$ represents the asymptotic time complexity of the recursive algorithm.

(c) Explain why $T(n)$ and $f_2(n)$ have the same asymptotic growth rate

From part 3, the exact running time is

$$T(n) = 3n \log_2 n + 9n - 5$$

We compare this with $f_2(n) = n \log_2 n$.

- The dominant term in $T(n)$ is $3n \log_2 n$.
- The terms $9n$ and -5 are of lower order than $n \log n$.
- Constant factors do not affect asymptotic growth.
- Therefore, $T(n)$ and $f_2(n)$ grow at the same rate.

$$T(n) = 3n \log_2 n + 9n - 5 = \Theta(n \log n) = \Theta(f_2(n))$$

7. Uneven Recursive Division

A second version of the algorithm splits a dataset of size n into subproblems of sizes $n/4$ and $3n/4$. The combining cost is $3n + 5$, and a dataset of size 1 costs 4. (Ignore rounding, so all sizes are exact.) Let the running time be $S(n)$.

(a) Recurrence relation

The algorithm makes two recursive calls on subproblems of sizes $n/4$ and $3n/4$, and then spends $3n + 5$ operations to combine the results. The base case is $S(1) = 4$.

Therefore, the recurrence is

$$S(n) = S\left(\frac{n}{4}\right) + S\left(\frac{3n}{4}\right) + 3n + 5, \quad n > 1$$

$$S(1) = 4$$

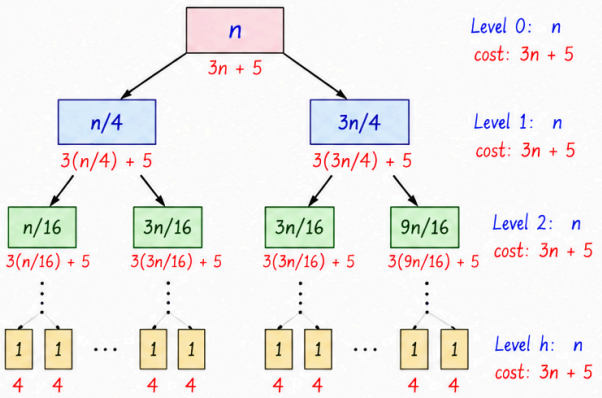
(b) Predict the asymptotic growth

The total size of the subproblems at each level is

$$\frac{n}{4} + \frac{3n}{4} = n.$$

Therefore, the non-recursive work at each level is $3n + 5 = \Theta(n)$.

Recursion tree (first few levels)



Although the tree is not perfectly balanced, the total size of subproblems at each level remains n, and each level costs $\Theta(n)$. The height is $\Theta(\log n)$. Therefore, the total work is $\Theta(n \log n)$.

$$S(n) = \Theta(n \log n).$$